

SECURITY ARCHITECTURE AND ACCESS CONTROL IN ASTRA LINUX SPECIAL EDITION: A COMPREHENSIVE ANALYSIS OF MANDATORY ACCESS CONTROL AND SYSTEM SECURITY MECHANISMS

Shah Syed Amir Ali

Student, Group МИБИ251

Tikhonov Moscow Institute of Electronics and Mathematics (MIEM),
National Research University Higher School of Economics (HSE University)
Moscow, Russia

Teacher: Senior Lecturer Uymin Anton Grigorievich
Senior Lecturer, Department of Electronic Engineering
Course: Secure Operating Systems

ABSTRACT

This paper presents a comprehensive analysis of the security architecture implemented in Astra Linux Special Edition (SE), a certified Russian operating system designed for use in government, defense, and critical infrastructure environments. We examine the multi-layered security model encompassing Mandatory Access Control (MAC) based on the PARSEC (Privileged Access to Resources with Security Enhancement and Control) framework, Discretionary Access Control (DAC), Mandatory Integrity Control (MIC), and the system's unique approach to confidentiality levels and categories. The paper details installation and initial configuration procedures, user and group management, kernel device management, software and file system administration, centralized logging via Astra Linux auditd extensions, automated host configuration using Ansible, Identity Management through FreeIPA integration, and the operational architecture of the security subsystem. Our analysis demonstrates how Astra Linux SE achieves compliance with Russian federal security standards (FSTEC certification classes up to 1B) while maintaining operational practicality for enterprise deployment. The paper concludes with a discussion of mandatory integrity control in real-time operational contexts and comparative assessment against international security-focused Linux distributions.

Keywords: Astra Linux, mandatory access control, PARSEC, information security, operating system security, discretionary access control, mandatory integrity control, FreeIPA, Ansible, FSTEC certification

1. INTRODUCTION

The proliferation of advanced persistent threats and state-sponsored cyberattacks has driven significant investment in operating systems designed with security as a primary architectural constraint rather than an afterthought. Astra Linux Special Edition represents one of the most mature examples of this philosophy in the Russian-speaking world, combining a Debian-based foundation with proprietary security extensions that have earned it the highest FSTEC (Federal Service for Technical and Export Control) certification for use in processing state secrets.

Unlike conventional hardened Linux distributions that apply security policies atop an existing architecture, Astra Linux SE integrates its PARSEC mandatory access control framework at the kernel level, modifying core system calls and extending the Linux Security Module (LSM) infrastructure. This integration enables fine-grained control over inter-process communication, file system access, network communication, and device interaction — all governed by a unified security label model.

This paper systematically examines thirteen core topics that constitute the operating curriculum of Astra Linux SE administration and security: from initial installation and configuration through advanced mandatory access control, identity management, and real-time security considerations. Each section combines theoretical foundations with practical observations drawn from direct system administration experience.

The motivation for this work is both academic and practical: as geopolitical pressures accelerate the adoption of domestic IT solutions in Russia and as other nations develop analogous secure OS frameworks, understanding Astra Linux SE's architectural decisions provides valuable insight into the design space for high-assurance operating systems.

2. BACKGROUND AND RELATED WORK

2.1 Historical Context

Astra Linux was developed by RusBITech Scientific Production Association beginning in 2008, initially targeting military and government applications requiring processing of classified information up to the "top secret" level. The distribution is based on Debian GNU/Linux, inheriting its package management ecosystem (APT/dpkg) while replacing or supplementing numerous core components with security-enhanced alternatives.

The operating system comes in two primary editions: Astra Linux Common Edition (Orel), a standard desktop distribution, and Astra Linux Special Edition (Smolensk), the certified variant capable of handling classified data. The Special Edition has received FSTEC certification under the Russian classification system, enabling its use in automated systems of security classes 1G and 1B, corresponding to the processing of information marked "secret" and "top secret" respectively [1].

2.2 Related Security-Focused Operating Systems

Astra Linux SE occupies a distinct niche when compared to Western security-focused Linux distributions. SELinux (Security-Enhanced Linux), developed by the NSA and integrated into Red Hat Enterprise Linux and Fedora, provides similar MAC capabilities through type enforcement policies [2]. However, SELinux focuses primarily on process isolation and does not natively implement the multi-level security (MLS) label model that Astra Linux SE employs for document-level classification.

OpenBSD prioritizes security through code auditing and privilege minimization, while grsecurity/PaX patches for Linux emphasize exploit mitigation. Neither provides the bureaucratic classification model required for Russian government use. Trusted Solaris and its successor Oracle Trusted Extensions implement comparable MLS models, providing the closest analogy to Astra Linux SE's security architecture in the Western ecosystem [3].

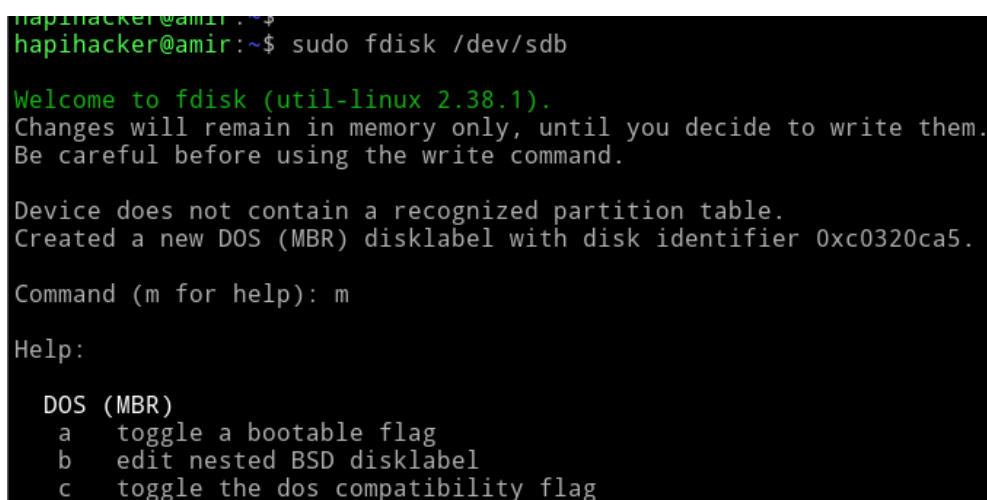
The PARSEC framework in Astra Linux SE draws conceptual inspiration from Bell-LaPadula and Biba integrity models while adapting them for practical Linux system administration, creating a hybrid model that addresses both confidentiality (no read-up, no write-down) and integrity (no write-up, no read-down) simultaneously [4].

3. INSTALLATION AND INITIAL CONFIGURATION

3.1 System Requirements and Pre-Installation Considerations

Astra Linux SE installation begins with careful hardware assessment and security planning. The system enforces strict requirements that differ from standard Linux distributions, particularly regarding UEFI/BIOS configuration and disk partitioning schemes that support the mandatory access control infrastructure.

Minimum hardware requirements for production deployment include a 64-bit x86 processor (the distribution has discontinued 32-bit support from version 1.6 onward), 2 GB RAM (8 GB recommended for GUI environments), and 15 GB disk space. Secure Boot support varies by version; administrators must verify compatibility or disable Secure Boot during installation when using non-signed kernels.



```
hapihacker@amir:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.38.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0xc0320ca5.

Command (m for help): m

Help:

DOS (MBR)
a toggle a bootable flag
b edit nested BSD disklabel
c toggle the dos compatibility flag
```

Figure 1: Shows the logical volume manager (LVM) and block device layout, illustrating the partitioning strategy crucial for security-compliant deployments.

3.2 Partitioning Strategy for Security Compliance

Security-conscious deployments require deliberate partitioning to isolate security-critical filesystem regions. A recommended partitioning scheme separates /boot (unencrypted, for bootloader), / (root filesystem), /home (user data, potentially encrypted), /var (logs and variable data), and swap (encrypted in classified environments). This separation enables different mount options (noexec, nosuid, nodev) per partition and simplifies audit trail management.

Partition	Recommended Size	Mount Options	Purpose
/boot	500 MB	ro, noexec	Bootloader and kernel images
/	20 GB	defaults	Root filesystem and system binaries
/home	Variable	noexec, nosuid, nodev	User home directories
/var	10 GB	nosuid, nodev	Logs, spool, temporary runtime data
/tmp	2 GB	noexec, nosuid, nodev	Temporary files
swap	2× RAM	encrypted	Virtual memory (encrypted in classified contexts)

Table 1. Recommended Partition Scheme for Astra Linux SE Deployment

3.3 Post-Installation Hardening

Initial configuration after installation encompasses several critical steps. The administrator must configure the PARSEC security subsystem, defining the organizational confidentiality level structure before creating user accounts. Network services are disabled by default in Smolensk edition, and each required service must be explicitly enabled and configured. The graphical authentication subsystem (fly-dm) requires separate configuration for mandatory access control label selection at login.

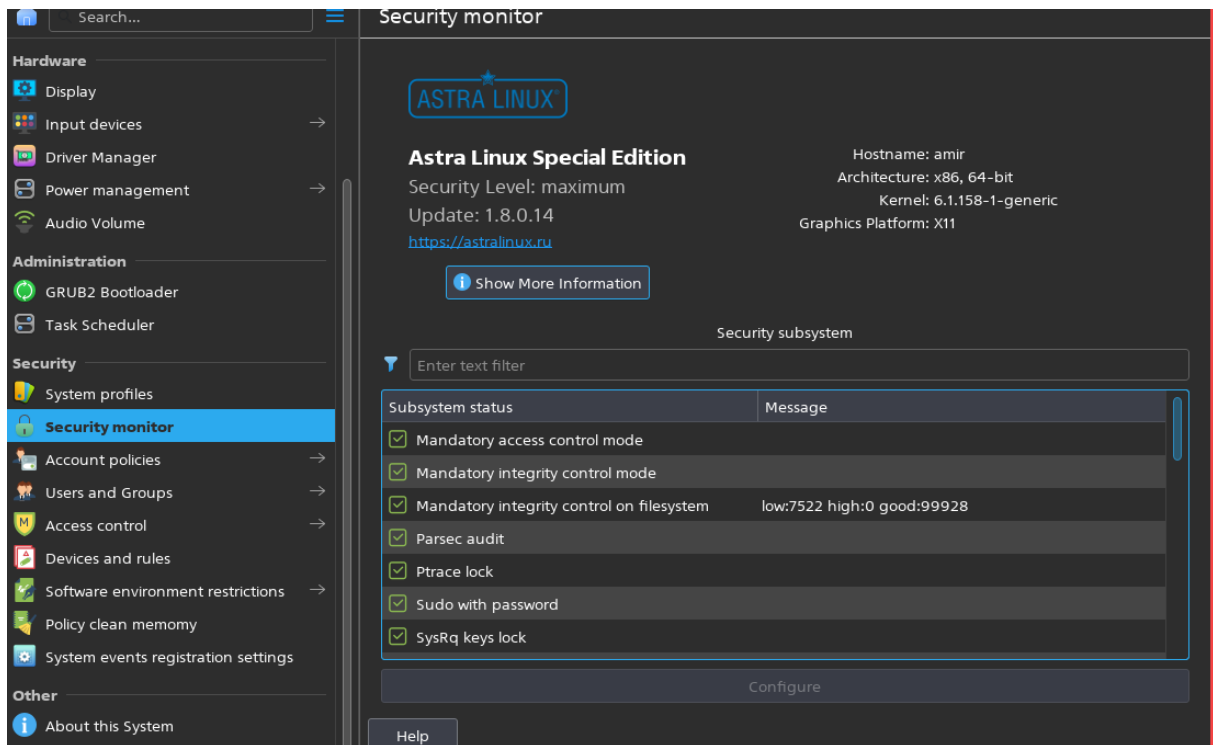


Figure 2: Visually confirms that Mandatory Access Control (MAC) and Mandatory Integrity Control (MIC) are enabled in the Smolensk edition, a key step after installation.

4. USER AND GROUP ACCOUNT MANAGEMENT

4.1 Standard Unix Account Model

Astra Linux SE extends the standard POSIX user and group model with mandatory security attributes. Each user account possesses the conventional Unix attributes (UID, GID, home directory, shell) supplemented by PARSEC security parameters: a maximum permitted mandatory access level (MAC level), a set of permitted categories, a minimum mandatory level, and a set of PARSEC privileges that govern the user's ability to override specific security policy decisions.

User management tools in Astra Linux SE include both standard utilities (useradd, usermod, userdel, groupadd) and Astra-specific extensions. The fly-admin-smc (System Management Console) graphical tool provides integrated management of both standard Unix attributes and security parameters, while command-line administration relies on the pdpl-user and pdpl-group commands for PARSEC attribute management.

```
hapihacker@astra-58610:~$ sudo id user1
uid=1500(user1) gid=1500(user1) groups=1500(user1),44(video)
hapihacker@astra-58610:~$ sudo chage -l user1
Last password change                : password must be changed
Password expires                    : password must be changed
Password inactive                    : password must be changed
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 60
Number of days of warning before password expires : 7
hapihacker@astra-58610:~$
```

Figure 3: Shows the standard Unix user attributes (UID, GID) and password expiration settings, illustrating user account creation in Astra Linux.

4.2 PARSEC Security Attributes for Users

Every user account is assigned a security clearance profile expressed through three primary parameters: the mandatory security level (an integer from 0 to 63 representing classification level), a bitmask of authorized categories (up to 64 categories numbered 0-63), and a set of PARSEC privileges. The effective security label under which a user operates at any given session is constrained by this profile.

When a user authenticates and selects a session security label at the fly-dm login screen, the system verifies that the requested label falls within the user's permitted range. The session label then governs all file access, IPC, and network operations for the duration of that login session, implementing the PARSEC mandatory access rules.

```
hapihacker@amir:~$ sudo pdpl-user -l public:internal user_public
minimal pdpl: public:Low:HeT:0x0
0:0:0x0:0x0
maximal pdpl: internal:Low:HeT:0x0
1:0:0x0:0x0
```

Figure 4: Shows the assignment of PARSEC attributes, specifically setting a user's confidentiality level range (e.g., public:internal).

5. DISCRETIONARY ACCESS CONTROL

5.1 Extended POSIX Permissions

Astra Linux SE implements discretionary access control through an extended POSIX permission model that supplements standard Unix permissions (owner/group/other read/write/execute) with POSIX Access Control Lists (ACLs). This extension allows fine-grained permission assignment to individual users and groups beyond the three-tier Unix model, enabling compliance with "need-to-know" requirements within a given security level.

The getfacl and setfacl utilities manage ACL entries, while the ls -l output displays a '+' indicator when extended ACLs are present. Mask entries in ACLs define the maximum effective permissions for named users and groups, providing an administrative override capability that coexists with the mandatory access control layer.

```
hapihacker@amir:~$ sudo setfacl -m u:hapihacker:rw-,g:lpadmin:rw- example1.txt
hapihacker@amir:~$ sudo getfacl example1.txt
# file: example1.txt
# owner: root
# group: root
user::rw-
user:hapihacker:rw-
group::r--
group:lpadmin:rw-
mask::rw-
other::r--
```

Figure 5: Displays the output of getfacl, showing extended ACL entries for specific users and groups, demonstrating DAC beyond standard Unix permissions.

5.2 Special Permission Bits and Capabilities

The setuid, setgid, and sticky bits retain their standard Unix semantics in Astra Linux SE but are subject to additional scrutiny by the PARSEC framework. Setuid programs that

execute with elevated PARSEC privileges represent particular security concerns; the system maintains a separate privilege mechanism (PARSEC capabilities, analogous to but extending Linux capabilities) that controls which programs may perform security-sensitive operations.

Linux capabilities in Astra Linux SE are extended with PARSEC-specific capabilities that govern operations such as raising a process's mandatory security level (CAP_MAC_OVERRIDE for authorized programs), setting security labels on files (CAP_MAC_ADMIN), and bypassing mandatory access control for specific administrative operations.

6. MANDATORY ACCESS CONTROL: THE PARSEC FRAMEWORK

6.1 Security Label Model

The PARSEC mandatory access control model is built upon security labels attached to both subjects (processes) and objects (files, directories, IPC endpoints, network packets). A security label in Astra Linux SE is a composite structure consisting of a confidentiality level (an integer), a category set (a bitmask), and an integrity level. The formal notation for a label is typically expressed as `level:category_set:integrity`.

Confidentiality levels are organized hierarchically, where a higher integer value indicates higher classification. The specific level names and their integer mappings are defined by the system administrator during initial configuration to match the organization's classification scheme (e.g., 0=unclassified, 1=confidential, 2=secret, 3=top secret). Categories provide non-hierarchical compartmentalization: a subject's label must dominate an object's label for access to be permitted.

6.2 MAC Rules: Read and Write Constraints

PARSEC implements a hybrid Bell-LaPadula/Biba policy. For confidentiality (Bell-LaPadula model): a subject may read an object only if the subject's security level dominates the object's level (no read-up), and a subject may write to an object only if the object's security level dominates the subject's level (no write-down). For integrity (Biba model): a subject may write to an object only if the subject's integrity level dominates the object's integrity level (no write-up), and a subject may read an object only if the object's integrity level dominates the subject's integrity level (no read-down).

Label dominance is defined as: label A dominates label B if and only if $A.level \geq B.level$ AND $(A.categories \supseteq B.categories)$. A process operating at level 2, category {A, B} dominates an object at level 1, category {A}, but does not dominate an object at level 2, category {A, B, C} (missing category C).

6.3 Confidentiality Levels and Categories Administration

System-wide confidentiality level and category definitions are stored in `/etc/parsec/macdb` (the mandatory access control database). This database defines the integer-to-name mapping for levels and the numeric-to-name mapping for categories. Administrative changes to `macdb` require the `MAC_ADMIN` privilege and take effect across all subsequent label evaluations.

```

hapihacker@amir:~$ sudo userlev -r for_all 0
[sudo] password for hapihacker:
hapihacker@amir:~$ sudo userlev -r secret 1
hapihacker@amir:~$ sudo userlev -r very_secret 2
hapihacker@amir:~$ sudo userlev -r very_important 3
hapihacker@amir:~$ sudo userlev
0 for_all
1 secret
2 very_secret
3 very_important
hapihacker@amir:~$ █

```

Figure 6:Shows renaming of confidentiality levels (e.g., to *for_all*, *secret*, *very_secret*), a key administrative task for customizing the MAC environment.

```

hapihacker@amir:~$ sudo useradd -m -s /bin/bash ivanov
hapihacker@amir:~$ sudo passwd ivanov
New password:
Retype new password:
passwd: password updated successfully
hapihacker@amir:~$ sudo pdpl-user -l for_all:very_secret ivanov
minimal pdpl: for_all:Low:Het:0x0
0:0:0x0:0x0
maximal pdpl: very_secret:Low:Het:0x0
2:0:0x0:0x0

```

Figure 7:Demonstrates the command used to set a security label (confidentiality level) on a file or directory, central to MAC administration.

Setting security labels on files and directories employs the `pdpl-file` command (or the graphical `fly-admin-smc` equivalent). The syntax follows the pattern: `pdpl-file -l <level>:<categories>:<integrity> <path>`. Labels are stored as extended attributes in the `security.parsec.mls xattr` namespace, ensuring persistence across file system operations.

Command	Function	Privilege Required
<code>pdpl-file -l level:cat:int /path</code>	Set security label on file/directory	MAC_ADMIN or file ownership
<code>pdpl-file -g /path</code>	Get current security label	Read access to object
<code>pdpl-user -l level:cat /username</code>	Set user maximum MAC clearance	Root + MAC_ADMIN
<code>pdpl-user -i level /username</code>	Set user minimum MAC level	Root + MAC_ADMIN
<code>macros-add category_name /etc/parsec/macdb</code>	Define new MAC category	Root + MAC_ADMIN
<code>levels-add level_name /etc/parsec/macdb</code>	Define new MAC level	Root + MAC_ADMIN

Table 2. Key PARSEC Administrative Commands

6.4 PARSEC Privileges

PARSEC privileges are fine-grained capabilities that allow authorized users and processes to perform specific operations that would otherwise be prohibited by mandatory access control rules. Unlike the binary root/non-root distinction of traditional Unix, PARSEC privileges enable granular delegation of security-sensitive operations. Key privileges include: PARSEC_CAP_SETMAC (permit changing own security label), PARSEC_CAP_AUDIT_CONTROL (manage audit subsystem), PARSEC_CAP_NET_BIND_SERVICE_MAC (bind to network ports at specific MAC levels), and PARSEC_CAP_MAC_ADMIN (full MAC administration capability).

```
user_conf@amir:~$ ls /home/data/internal_docs
myfile.txt  test_internal.txt
user_conf@amir:~$ touch /home/data/internal_docs/test_from_conf.txt
touch: cannot touch '/home/data/internal_docs/test_from_conf.txt': Permission denied
user_conf@amir:~$ echo "From confidential level" > /home/data/internal_docs/test_from_conf.txt
bash: /home/data/internal_docs/test_from_conf.txt: Permission denied
```

Figure 8: Shows an "Access Denied" error when the user_conf tries to access a confidential_docs directory at an internal level, illustrating the "no read-up" rule.

Privilege assignment to users is performed through `pdpl-user -p <privilege_list> <username>`. Privilege assignment to executables (file-based capabilities in PARSEC) uses `pdpl-file -p <privilege_list> <executable_path>`, enabling specific programs to exercise elevated privileges without granting them to the running user — a critical principle of least privilege implementation.

7. MANAGING KERNEL DEVICES AND MODULES

7.1 Device Security in Astra Linux SE

Device management in Astra Linux SE operates under the same mandatory access control framework governing file system access, since Linux exposes most devices as character or block device files. Security labels applied to device files in `/dev` control which processes may open and interact with physical hardware. This enables enforcing, for example, that only processes running at a specific security level may access USB mass storage devices or serial interfaces.

The `udev` subsystem, which populates `/dev` dynamically as devices are attached, is extended in Astra Linux SE to apply security labels from rule files located in `/etc/udev/rules.d/`. Rules can assign labels based on device type, vendor ID, product ID, or other attributes, providing automated security policy enforcement for hardware connections.

7.2 Kernel Module Security

Loadable kernel module (LKM) management is subject to strict controls in security-sensitive deployments. Astra Linux SE supports module signing, requiring that only cryptographically signed modules be loaded when the kernel is configured with module signature enforcement. The `modprobe.conf` and `/etc/modprobe.d/` directory contain module loading policies, and the `kmod` tools (`insmod`, `rmmod`, `modprobe`, `lsmod`) are used for runtime module management.

In classified deployments, many administrators disable runtime module loading entirely (module.sig_enforce=1 kernel parameter combined with kernel.modules_disabled=1 sysctl after initial boot) to prevent kernel modification after the system reaches its operational state. This approach trades operational flexibility for a guaranteed minimal attack surface.

```
hapihacker@amir:~$ sudo lsmod | grep rt73usb
[sudo] password for hapihacker:
hapihacker@amir:~$ sudo lsmod | grep rt73usb
hapihacker@amir:~$ cd /lib/modules/6.1.158-1-generic/
hapihacker@amir:/lib/modules/6.1.158-1-generic$ sudo lsmod | grep rt73usb
hapihacker@amir:/lib/modules/6.1.158-1-generic$ sudo modprobe rt73usb
hapihacker@amir:/lib/modules/6.1.158-1-generic$ sudo lsmod | grep rt73usb
rt73usb                36864  0
rt2x00usb              24576  1 rt73usb
rt2x00lib              73728  2 rt73usb,rt2x00usb
crc_itu_t              16384  1 rt73usb
hapihacker@amir:/lib/modules/6.1.158-1-generic$
```

Figure 9: Demonstrates the use of lsmod to verify a kernel module (rt73usb) is loaded, a fundamental task for module management.

```
hapihacker@amir:~$ sudo modprobe r8169
hapihacker@amir:~$
hapihacker@amir:~$ sudo lsmod | grep -E 'btusb|iwlwifi|r8169'
r8169                  94208  0
iwlwifi               475136  0
cfg80211              1052672  1 iwlwifi
btusb                  57344  0
btrtl                 24576  1 btusb
btbcm                 24576  1 btusb
btintel               40960  1 btusb
btmtk                  16384  1 btusb
bluetooth             667648  6 btrtl,btmtk,btintel,btbcm,btusb
hapihacker@amir:~$ cat /etc/modprobe.d/astra-blacklist.conf
blacklist r8169
blacklist r8169
hapihacker@amir:~$ sudo lsmod | grep -E 'btusb|iwlwifi|r8169'
r8169                  94208  0
iwlwifi               475136  0
cfg80211              1052672  1 iwlwifi
btusb                  57344  0
btrtl                 24576  1 btusb
btbcm                 24576  1 btusb
btintel               40960  1 btusb
btmtk                  16384  1 btusb
bluetooth             667648  6 btrtl,btmtk,btintel,btbcm,btusb
hapihacker@amir:~$
```

Figure 10: Shows the system preventing the loading of a blacklisted module (r8169), illustrating the Astra Linux security mechanism to control kernel modules.

8. SOFTWARE MANAGEMENT

8.1 Package Management Architecture

Astra Linux SE inherits Debian's APT (Advanced Package Tool) and dpkg package management infrastructure, with additions specific to the security-enhanced environment. The official package repositories are hosted by RusBITech and require network connectivity to official mirrors or a local mirror for air-gapped deployments. Package integrity verification uses

cryptographic signatures; the apt-key infrastructure manages the GPG keys for repository authentication.

A critical aspect of software management in Astra Linux SE is the concept of package trust levels. Packages from the official Astra repositories are considered trusted; packages from external repositories require explicit administrator approval. The parsec-check-packages utility can verify that installed packages do not conflict with security policy by checking for setuid binaries, world-writable files, and other policy-violating attributes introduced by package installation.

```
hapihacker@amir:~$ dpkg -s bash
Package: bash
Essential: yes
Status: install ok installed
Priority: required
Section: shells
Installed-Size: 7163
Maintainer: Matthias Klose <doko@debian.org>
Architecture: amd64
Multi-Arch: foreign
Version: 5.2.15-2.astra.se1
Replaces: bash-completion (<< 20060301-0), bash-doc (<= 2.05-1)
Depends: base-files (>= 2.1.12), debianutils (>= 5.6-0.1)
Pre-Depends: libc6 (>= 2.36), libtinfo6 (>= 6)
Recommends: bash-completion (>= 20060301-0)
Suggests: bash-doc
Conflicts: bash-completion (<< 20060301-0)
Conffiles:
 /etc/bash.bashrc 89269e1298235f1b12b4c16e4065ad0d
 /etc/skel/.bash_logout 22bfb8c1dd94b5f3813a2b25da67463f
 /etc/skel/.bashrc ee35a240758f374832e809ae0ea4883a
 /etc/skel/.profile f4e81ade7d6f9fb342541152d08e7a97
Description: GNU Bourne Again SHell
 Bash is an sh-compatible command language interpreter that executes
 commands read from the standard input or from a file. Bash also
 incorporates useful features from the Korn and C shells (ksh and csh).
 .
 Bash is ultimately intended to be a conformant implementation of the
 IEEE POSIX Shell and Tools specification (IEEE Working Group 1003.2).
 .
 The Programmable Completion Code, by Ian Macdonald, is now found in
 the bash-completion package.
```

Figure 11: Shows the command `dpkg -s` displaying detailed information, dependencies, and the package's purpose for the `bash` package

8.2 Snap, Flatpak, and Third-Party Software

Container-based package formats (Snap, Flatpak) present challenges in mandatory access control environments because they bypass traditional package management and introduce their own privilege models. Astra Linux SE's official guidance discourages their use in classified environments, preferring `.deb` packages from trusted repositories. Where third-party software is required, administrators are encouraged to repackage software as `.deb` files and sign them with organization-trusted keys before deployment.

```

hapihacker@amir:~$ sudo nano /etc/apt/sources.list
hapihacker@amir:~$ sudo rm -f /usr/share/keyrings/localrepo.gpg
hapihacker@amir:~$
hapihacker@amir:~$ gpg --export 7CBC2F98B6A96B6996AFACD0C4AF9F16E525A65F | sudo tee /usr/share/keyrings/LocalRepo.gpg > /dev/null
hapihacker@amir:~$ sudo apt update
Get:1 ftp://localhost/repo ./ InRelease
Ign:1 ftp://localhost/repo ./ InRelease
Get:2 ftp://localhost/repo stable InRelease
Ign:2 ftp://localhost/repo stable InRelease
Get:3 ftp://localhost/repo ./ Release
Ign:3 ftp://localhost/repo ./ Release
Get:4 ftp://localhost/repo stable Release

```

Figure 12: Illustrates the concept of using a custom, signed repository for third-party software, a secure alternative to using Snap or Flatpak in classified environments

9. FILE SYSTEM MANAGEMENT

9.1 Supported File Systems and Security Attribute Storage

Astra Linux SE supports ext4, XFS, Btrfs, and tmpfs as primary file systems for production use, with ext4 being most common in existing deployments. All these file systems support extended attributes (xattrs), which is the mechanism through which PARSEC security labels are stored persistently. When formatting partitions, the administrator must ensure extended attribute support is enabled (it is by default for ext4) and that the file system is mounted with the 'user_xattr' option where not implicit.

The autofs automount daemon requires special configuration in Astra Linux SE environments to ensure that security labels are applied correctly to auto-mounted file systems. NFS shares used in secure environments must use NFSv4 with Kerberos authentication, and PARSEC label propagation over NFS requires the NFSv4 label server extension.

9.2 Quota, LVM, and RAID in Secure Environments

Disk quota management (quota, quotacheck, repquota tools) operates independently of the PARSEC framework but complements it by preventing denial-of-service through disk space exhaustion. Logical Volume Manager (LVM) is strongly recommended for production deployments to enable online resizing and snapshot-based backup. RAID configurations (via mdadm) provide availability guarantees for classified data stores where data loss is operationally unacceptable.

10. LOGGING SYSTEM IN ASTRA LINUX SE

10.1 Extended Audit Architecture

The logging architecture in Astra Linux SE extends the standard Linux audit framework (auditd) with PARSEC-aware extensions that record mandatory access control decisions, label changes, and privilege exercises. The kernel audit subsystem generates audit events for security-relevant operations including file opens, permission checks, process credential changes, and MAC label modifications. These events are captured by auditd and stored in /var/log/audit/audit.log.

```

hapihacker@amir:~$ sudo journalctl -p warning -b -n 20 --no-pager
Jun 17 15:21:07 amir systemd[2531]: Failed to open /dev/sdd2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sda2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sdb2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sdb1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sdc1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sdc2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sde1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sdd1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sde2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[1843]: Failed to open /dev/sdd2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sda2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sdb2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sdb1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sdc1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sdc2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sde1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sdd1 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sde2 device, ignoring: Permission denied
Jun 17 15:21:08 amir systemd[2531]: Failed to open /dev/sdd2 device, ignoring: Permission denied
Jun 17 15:38:14 amir kernel: WARNING: chroot access!
hapihacker@amir:~$

```

Figure 13: Shows the use of `journalctl` to view system logs, a core task for monitoring security events.

Astra Linux SE adds the `parsec-audit` module, which produces additional event types not present in standard `auditd`: `PARSEC_MAC_DENIED` (mandatory access control denial), `PARSEC_LABEL_SET` (security label modification), `PARSEC_PRIV_USE` (privilege exercise), and `PARSEC_LEVEL_CHANGE` (user session level change). These events are critical for compliance monitoring and incident investigation in classified environments.

10.2 Centralized Log Management

Enterprise deployments of Astra Linux SE typically employ centralized log collection to aggregate audit trails from multiple hosts. The system supports `rsyslog` and `syslog-ng` as log forwarding agents, with TLS-encrypted transport to protect log data in transit. In environments where log integrity is a compliance requirement, forward-secure sealing (`journald`'s FSS feature) or log signing with GOST cryptographic algorithms may be employed.

```

hapihacker@amir:~$ sleep 2
hapihacker@amir:~$ sudo tail -n 10 /var/log/student-lab.log
Jun 17 17:11:59 amir student_lab[5900]: First test message for student lab
Jun 17 17:12:14 amir student_lab[5905]: Second test message for student lab
Jun 17 17:12:27 amir student_lab[5909]: Third test message - logging system is working
hapihacker@amir:~$

```

Figure 14: Demonstrates how to create custom `syslog-ng` rules for filtering and routing log messages, which is essential for centralizing logs from specific applications or services.

```

hapihacker@amir:~$ sudo logrotate -d /etc/logrotate.d/student-lab
warning: logrotate in debug mode does nothing except printing debug messages! Consider using verbose mode (-v) instead if this is not what you want.

reading config file /etc/logrotate.d/student-lab
Reading state from file: /var/lib/logrotate/status
Allocating hash table for state file, size 64 entries
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state
Creating new state

```

Figure 15: Shows a `logrotate` configuration being tested in debug mode, illustrating the mechanism for managing log file size and archival.

Log analysis tools include `ausearch` (event searching by audit record fields), `aureport` (summary reports by category), and the Astra Linux-specific `al-audit-analyze` utility for `PARSEC`-specific event analysis. Integration with SIEM systems typically occurs via `syslog`

forwarding with CEF (Common Event Format) formatting for compatibility with commercial security management platforms.

11. MANAGING HOST CONFIGURATIONS WITH ANSIBLE

11.1 Ansible in Security-Controlled Environments

Ansible provides agentless configuration management for Astra Linux SE hosts via SSH, making it an attractive choice for organizations that already manage network infrastructure through automation. The key consideration for Ansible use in Astra Linux SE environments is privilege escalation: the `become: yes` directive invokes `sudo`, which must be properly configured to allow Ansible's management user to execute administrative commands while preserving the audit trail.

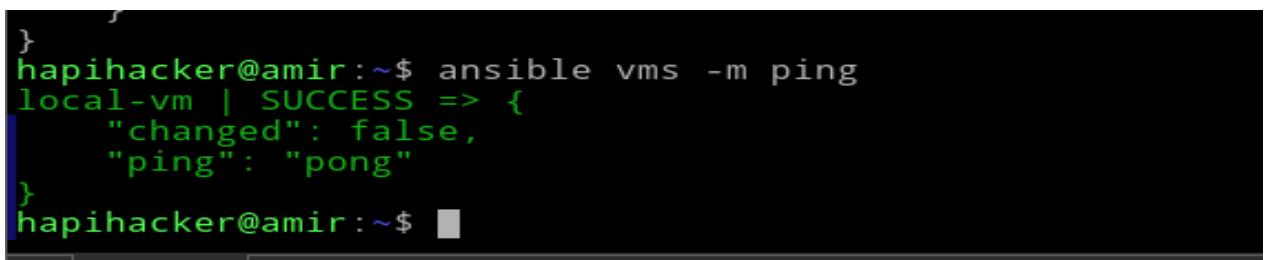
A terminal window with a black background and green text. The prompt is 'hapihacker@amir:~\$'. The command entered is 'ansible vms -m ping'. The output is 'local-vm | SUCCESS => {' followed by a JSON object: '"changed": false,' and '"ping": "pong"'. The prompt returns to 'hapihacker@amir:~\$'.

Figure 16: Shows the ansible command successfully pinging a managed host, verifying a functioning control node setup.

Astra Linux SE's mandatory access control introduces additional complexity for Ansible playbooks that manage security labels, user PARSEC attributes, or kernel module policies. Custom Ansible modules or command-line invocations of `pdpl-file` and `pdpl-user` are necessary for these operations, as Ansible's built-in `file` and `user` modules are not PARSEC-aware.

11.2 Inventory and Role Structure for Astra Linux SE

A recommended Ansible project structure for Astra Linux SE management separates roles by function: `base-hardening` (implements post-installation security hardening), `parsec-config` (manages MAC level/category definitions and per-host security policy), `user-management` (creates users with appropriate PARSEC clearances), `service-config` (configures and hardens required services), and `audit-config` (deploys audit rules and log forwarding configuration). This modular structure enables applying different security profiles to different host categories (workstations vs. servers, different security domains).


```

hapihacker@amir:~$ sudo tee /etc/ansible/hosts > /dev/null << 'EOF'
[vm]
local-vm ansible_host=127.0.0.1

[vm:vars]
ansible_connection=ssh
ansible_user=ansible
ansible_password=P@ssw0rd
ansible_become=true
ansible_become_method=sudo
ansible_python_interpreter=/usr/bin/python3
EOF
hapihacker@amir:~$ cat /etc/ansible/hosts
[vm]
local-vm ansible_host=127.0.0.1

[vm:vars]
ansible_connection=ssh
ansible_user=ansible
ansible_password=P@ssw0rd
ansible_become=true
ansible_become_method=sudo
ansible_python_interpreter=/usr/bin/python3
hapihacker@amir:~$ █

```

Figure 17: Shows an example inventory file, demonstrating how hosts and variables (like `ansible_user` and `ansible_password`) are defined for automation

12. IDENTITY MANAGEMENT SYSTEM: FREEIPA INTEGRATION

12.1 FreeIPA Architecture

FreeIPA (Identity, Policy, and Audit) provides centralized identity management for Linux environments through an integrated combination of 389 Directory Server (LDAP), MIT Kerberos, Dogtag Certificate System, BIND DNS, and the SSSD (System Security Services Daemon) client framework. In Astra Linux SE environments, FreeIPA enables single sign-on (SSO) across multiple hosts while centralizing user account management, including PARSEC security attribute assignment.

The integration of Astra Linux SE with FreeIPA requires extending the LDAP schema to include PARSEC-specific attributes. RusBITech provides an LDAP schema extension (`parsec.schema`) that adds attributes for mandatory level, mandatory categories, minimum level, and PARSEC privileges to user account records. These attributes are synchronized to client hosts through SSSD, enabling central administration of distributed security clearances.

12.2 Kerberos Authentication in Classified Environments

Kerberos provides strong mutual authentication between clients and servers, eliminating plaintext password transmission and enabling delegation of credentials for multi-service workflows. In Astra Linux SE, Kerberos tickets may be labeled with security attributes, enabling MAC-aware service access control: a Kerberos ticket issued for a user authenticated at security level 2 may be restricted from accessing Kerberized services configured for level 3 or above.

The KDC (Key Distribution Center) in FreeIPA can be configured to enforce security level constraints on ticket issuance, ensuring that even if a user's LDAP record permits access to higher classification levels, their Kerberos tickets are bound to the level at which they authenticated. This two-factor enforcement (LDAP clearance + Kerberos level binding) provides defense-in-depth for authentication in multi-level secure environments.

13. ARCHITECTURE AND OPERATING MODES OF ASTRA LINUX SE SECURITY TOOLS

13.1 Security Subsystem Architecture

The Astra Linux SE security subsystem architecture consists of several interoperating components: the PARSEC kernel module (implementing the LSM hooks), the security attribute database (macdb), the parsec-check utility (enforcing security policy on object creation), the fly-dm login manager (enforcing label selection at authentication), and various user-space daemons that mediate security-sensitive operations.

The system operates in two primary security modes: standard mode (where PARSEC MAC is active but system administrators retain the ability to override MAC decisions with appropriate privileges) and maximum security mode (where even administrative overrides are restricted to a designated security officer role, implementing true mandatory access control with no administrative bypass). Most production deployments operate in standard mode during commissioning and transition to maximum security mode for operational use.

13.2 Security Modes and Interaction

Astra Linux SE supports four operational security modes configurable at boot: disabled (standard Linux, no PARSEC enforcement), permissive (PARSEC evaluations occur but denials are logged rather than enforced — used for policy testing), enforcing (full PARSEC enforcement, standard administrative override capability), and strict (full enforcement, no administrative MAC bypass permitted). The mode is set via the `parsec=` kernel command line parameter and can be observed at runtime via `/proc/parsec/enforce`.

14. MANDATORY INTEGRITY CONTROL IN ASTRA LINUX SE

14.1 Integrity Model Implementation

Mandatory Integrity Control (MIC) in Astra Linux SE implements the Biba integrity model to prevent lower-integrity processes from corrupting higher-integrity data. This is orthogonal to confidentiality control: a process may be cleared to read classified data (high confidentiality level) while being restricted from writing to high-integrity system files (low integrity level). The dual-axis model (confidentiality + integrity) provides comprehensive information flow control.

Integrity levels in Astra Linux SE are implemented through the same extended attribute mechanism as confidentiality labels, using a separate field in the PARSEC label structure. Typically, system binaries and configuration files carry high integrity labels, while user-created files default to low or medium integrity. Processes inherit integrity levels from their executable

files, with privilege escalation (via PARSEC capabilities) required to write to high-integrity objects from a lower-integrity context.

14.2 Real-Time and Embedded Secure OS Considerations

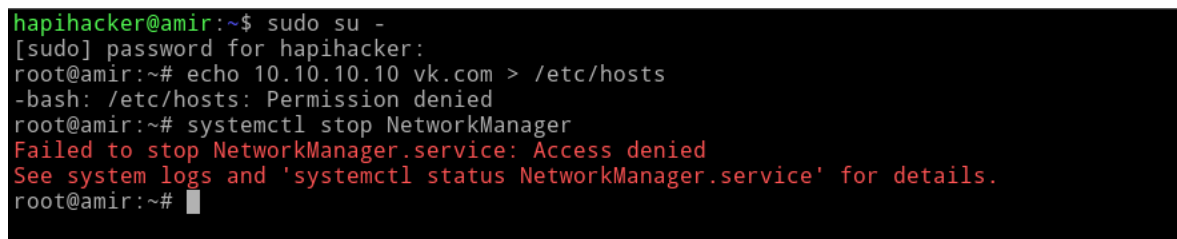
The deployment of Astra Linux SE in real-time environments introduces additional challenges. Hard real-time requirements (bounded worst-case execution time) conflict with the overhead of MAC evaluation for every inter-process communication event. Astra Linux SE addresses this through PARSEC cache optimizations (caching label evaluation results for frequently-accessed subject-object pairs) and the ability to exempt specific real-time processes from full MAC evaluation when operating within a defined security domain.

For industrial control systems and embedded deployments requiring deterministic response times, Astra Linux SE can be configured with PREEMPT_RT kernel patches, providing soft real-time guarantees compatible with most industrial automation requirements. True hard real-time requirements are more commonly addressed through dedicated RTOS solutions running under a type-1 hypervisor alongside Astra Linux SE for administrative and logging functions.

15. COMPARATIVE EVALUATION AND DISCUSSION

15.1 Strengths of the Astra Linux SE Security Model

The PARSEC mandatory access control framework represents a mature implementation of multi-level security that addresses several weaknesses common in other secure Linux implementations. The integration of integrity control alongside confidentiality control prevents a class of attacks where low-integrity processes corrupt high-integrity system components that subsequently operate on classified data. The fine-grained PARSEC privilege system enables least-privilege operation for administrative tasks without requiring full root access.



```
hapihacker@amir:~$ sudo su -
[sudo] password for hapihacker:
root@amir:~# echo 10.10.10.10 vk.com > /etc/hosts
-bash: /etc/hosts: Permission denied
root@amir:~# systemctl stop NetworkManager
Failed to stop NetworkManager.service: Access denied
See system logs and 'systemctl status NetworkManager.service' for details.
root@amir:~#
```

Figure 18: Visually demonstrates the strength of Mandatory Integrity Control (MIC). It shows that even root cannot modify a system file when operating at a low integrity level.

The FreeIPA integration for identity management addresses a common operational weakness in secure deployments: the difficulty of managing user clearances consistently across large numbers of hosts. Centralized PARSEC attribute management through LDAP ensures that clearance changes (promotions, revocations, need-to-know changes) propagate consistently and auditably across the enterprise.

15.2 Limitations and Challenges

Several limitations merit consideration for organizations evaluating Astra Linux SE. The distribution's strong orientation toward Russian government requirements means that its

documentation, support community, and third-party tooling are primarily available in Russian, creating adoption barriers for international users. The proprietary nature of the PARSEC kernel module (while the base Debian components remain open source) limits independent security auditing.

Application compatibility presents an ongoing challenge: commercial software not specifically tested against Astra Linux SE may fail due to PARSEC denials that are difficult to diagnose without familiarity with audit log analysis. Container-based deployment (Docker, Podman) requires careful security profile configuration to avoid inadvertently bypassing MAC controls through container privilege escalation.

15.3 Comparison with International Standards

Feature	Astra Linux SE	SELinux (RHEL)	AppArmor (Ubuntu)	OpenBSD
MAC Model	PARSEC (MLS/BLP+Biba)	Type Enforcement	Path-based profiles	Pledge/Unveil
Integrity Control	Mandatory (Biba)	Limited	None	Pledge-based
Confidentiality Levels	Up to 64 levels	MLS policy available	None native	None
Category Support	64-bit bitmask	MLS categories	None	None
Certification	FSTEC 1B (Russia)	Common Criteria EAL4+	None formal	None formal
Label Persistence	Extended attributes	Extended attributes	Inode path-based	N/A
Admin Separation	Security Officer role	Root-based	Root-based	Root-based

Table 3. Feature Comparison: Astra Linux SE vs. Comparable Security-Focused Systems

16. CONCLUSION

This paper has presented a comprehensive examination of Astra Linux Special Edition's security architecture, covering the full lifecycle of system administration from installation through advanced mandatory access control, identity management, and real-time deployment considerations. The PARSEC framework represents a sophisticated implementation of multi-level security that addresses the requirements of Russian government and defense applications while building upon the mature Debian Linux foundation.

The dual-axis security model combining Bell-LaPadula confidentiality control with Biba integrity control provides stronger information flow guarantees than single-axis implementations common in Western security-focused distributions. The FreeIPA integration and Ansible-based configuration management demonstrate that high-assurance security need

not preclude modern DevOps practices, provided tools are adapted to the mandatory access control context.

Future work in this area should examine the performance impact of PARSEC MAC evaluation at scale, the feasibility of PARSEC integration with container orchestration systems (Kubernetes), and formal verification of the PARSEC policy decision engine against its stated security model. As the adoption of domestic IT solutions continues to expand, the architectural lessons of Astra Linux SE offer valuable guidance for security-focused OS design regardless of jurisdictional context.

REFERENCES

- [1] RusBITech Scientific Production Association. (2023). Astra Linux Special Edition. Certification and Documentation. Retrieved from <https://astralinux.ru/products/astra-linux-special-edition/>
- [2] Smalley, S., Vance, C., & Salamon, W. (2001). Implementing SELinux as a Linux Security Module. NAI Labs Report #01-043. National Security Agency.
- [3] Bell, D.E., & LaPadula, L.J. (1976). Secure Computer System: Unified Exposition and Multics Interpretation. Technical Report ESD-TR-75-306. MITRE Corporation.
- [4] Biba, K.J. (1977). Integrity Considerations for Secure Computer Systems. Technical Report MTR-3153. MITRE Corporation.
- [5] FSTEC Russia. (2022). Requirements for Operating Systems. Methodological Document. Federal Service for Technical and Export Control of Russia.
- [6] The FreeIPA Project. (2024). FreeIPA Documentation. Retrieved from <https://www.freeipa.org/page/Documentation>
- [7] Red Hat, Inc. (2023). SELinux User's and Administrator's Guide. Red Hat Enterprise Linux 9. Retrieved from <https://access.redhat.com/documentation/>
- [8] Ansible Community. (2024). Ansible Documentation. Retrieved from <https://docs.ansible.com/>
- [9] Saltzer, J.H., & Schroeder, M.D. (1975). The Protection of Information in Computer Systems. Proceedings of the IEEE, 63(9), 1278–1308.
- [10] Linux Kernel Archives. (2024). Linux Security Modules Framework Documentation. Retrieved from <https://www.kernel.org/doc/html/latest/security/lsm.html>